2025 IEEE CHILEAN Conference on Electrical, Electronics Engineering, Information and Communication Technologies (CHILECON) | 979-8-3503-5736-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/CHILECON66915.2025.11476563

Leveraging LLMs for Functional Clustering of Microservices

1st Santiago Di Sabatto
UNCPBA
Tandil, Argentina
santiagodisabatto@gmail.com

2nd Guillermo Rodriguez
ISISTAN(UNICEN-CONICET)
Tandil, Argentina
grodri@exa.unicen.edu.ar

3rd Claudia Marcos
ISISTAN(UNICEN-CONICET)
Tandil, Argentina
cmarcos@exa.unicen.edu.ar

4th Santiago Vidal
ISISTAN(UNICEN-CONICET)
Tandil, Argentina
svidal@exa.unicen.edu.ar

***Abstract* — Microservices architectures often suffer from fragmentation and complexity, making system maintenance, scalability, and evolution increasingly difficult. A key challenge in this context is how to effectively group microservices into coherent functional modules. This paper proposes a novel approach for the functional grouping of microservices using Large Language Models (LLMs) and clustering techniques. Our methodology combines semantic and structural analysis of microservices based on their OpenAPI specifications and was validated using real-world microservice datasets. The results demonstrate that integrating semantic and structural similarities leads to more coherent and manageable groupings. Furthermore, leveraging ChatGPT to generate missing endpoint descriptions significantly improved clustering accuracy, achieving a 60% higher success rate compared to methods without automatic description generation. Our findings suggest that the proposed approach not only enhances the maintainability and scalability of microservice architectures but also streamlines development and deployment workflows, offering a scalable and efficient solution for service organization.**

***Index Terms*—microservices, OpenAPI, LLMs, ChatGPT, software architecture, clustering.**

I. Introducción

En el contexto de las aplicaciones web, un microservicio se define como una unidad de software que cumple una función específica dentro de la aplicación [1]. Cada microservicio opera de manera independiente y autónoma, ejecutándose en su propio servidor [1]. Esto significa que cada microservicio es capaz de funcionar de forma aislada, sin depender directamente de otros componentes de la aplicación. Entonces, es así que una aplicación web está compuesta por múltiples microservicios que se comunican entre sí.

Los microservicios están ganando cada vez más relevancia en el contexto de las aplicaciones web debido a su capacidad para proporcionar una alta flexibilidad en la adaptación a los cambios constantes de los requisitos de los usuarios y la evolución tecnológica [2, 4]. No obstante, se presenta una problemática general en lo que respecta a su distribución en diferentes servidores, el escenario ideal implica que cada microservicio se ejecute de manera aislada en su propio servidor.

En consecuencia, este trabajo propone un enfoque denominado AMDO (Agrupamiento de Microservicios en la Documentación OpenAPI). AMDO analiza las especificaciones OpenAPI de microservicios, agrupándolos y asignándolos a diferentes servidores o clústeres. La clave de este proceso es la interpretación de los endpoints en las documentaciones OpenAPI para permitir que microservicios con similitudes semánticas y estructurales se agrupen en el mismo clúster. AMDO utiliza un modelo de grandes lenguajes, como Chat GPT, para la identificación de microservicios a partir de las especificaciones OpenAPI, y el algoritmo de clustering k-means para agrupar microservicios en clusters.

Este trabajo presenta **dos contribuciones clave** a la investigación en el área de microservicios y procesamiento de lenguaje natural. En **primer lugar**, proponemos el enfoque AMDO (Agrupamiento de Microservicios en la Documentación OpenAPI), un proceso automatizado que utiliza modelos de lenguaje y algoritmos de clustering para agrupar microservicios en función de sus similitudes semánticas y estructurales. AMDO facilita la organización, mantenimiento y escalabilidad de arquitecturas de microservicios, lo que representa una mejora significativa frente a enfoques manuales o semiautomatizados. En **segundo lugar**, este trabajo analiza el uso de ChatGPT como herramienta de asistencia en el proceso de generación automática de descripciones faltantes en la documentación OpenAPI [3]. Evaluamos cómo la capacidad de ChatGPT para generar texto coherente y relevante puede enriquecer el análisis de microservicios, mejorando la calidad del agrupamiento semántico. Este análisis no solo demuestra la eficacia de ChatGPT en tareas de apoyo al desarrollo de software, sino que también introduce nuevas posibilidades para su aplicación en otros contextos de automatización de procesos.

El resto del trabajo se organiza de la siguiente manera. La Sección II compara los trabajos relacionados. La Sección III presenta nuestro enfoque para identificar microservicios. La Sección IV describe el diseño del experimento. La Sección V muestra los resultados obtenidos. La Sección VI presenta la discusión y responde las preguntas de investigación. La Sección VII detalla las limitaciones de nuestro enfoque. Finalmente, la Sección VIII concluye el trabajo e identifica futuras líneas de trabajo.

II. Trabajos Relacionados

Diversos trabajos han abordado el agrupamiento de microservicios según sus similitudes funcionales y estructurales. El trabajo de [17] propone descomponer aplicaciones monolíticas en microservicios mediante análisis semántico y clustering jerárquico, utilizando embeddings como fastText y Word2Vec sobre especificaciones OpenAPI. A diferencia de otros enfoques, identifica automáticamente la cantidad de clusters. En [5], se destacan los desafíos en la definición de interfaces y granularidad adecuada en arquitecturas de microservicios, utilizando análisis semántico de OpenAPI para detectar microservicios candidatos. El estudio de [19] introduce métricas como SGM, LCOM y NOO para evaluar granularidad, cohesión y complejidad, también desde una perspectiva de descomposición monolítica. Cabe destacar que estos trabajos operan sobre aplicaciones monolíticas ya existentes, mientras que nuestra propuesta analiza microservicios de forma aislada. Finalmente, [21] emplea modelado de tópicos (LDA, LSI, NMF) para inferir la funcionalidad y agrupar APIs en contenedores, resaltando la importancia de descripciones en OpenAPI. Si bien este enfoque no parte de

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 27,2026 at 23:22:03 UTC from IEEE Xplore. Restrictions apply.

una monolítica, no integra simultáneamente similitudes funcionales y estructurales como nuestro enfoque.

III. AMDO - Proceso de Identificación de Microservicios

AMDO (Agrupamiento de Microservicios en la Documentación OpenAPI), se centra en analizar y organizar la información textual de las descripciones de los diversos métodos en la documentación OpenAPI relacionada con cada microservicio. Luego, estas descripciones se agrupan de manera que cada grupo representa claramente la funcionalidad y las características de las descripciones que contiene. Finalmente, se utiliza un algoritmo para identificar patrones comunes en estos grupos y agrupar los microservicios en función de sus similitudes funcionales.

AMDO consta de varias actividades que conducen al resultado deseado (Figura 1). En primer lugar, **identificar y extraer las descripciones de los microservicios**, y en caso de no encontrar alguna, se **genera de forma automática** con la información provista por el endpoint. A continuación, **procesar las descripciones** para extraer las partes más importantes con el fin de poder realizar un mejor análisis de las funcionalidades de las mismas (sub-sección A). Estas etapas iniciales son críticas ya que proporcionan los datos de entrada al sistema.

Una vez que se ha recopilado la información completa, se procede a **identificar estructuras y funcionalidades similares** entre las diferentes descripciones para así poder formar diferentes agrupamientos (sub-sección B). Luego de formados estos grupos, se utilizan para **generar una representación única para cada documento**, que refleja las funcionalidades inherentes a dicho documento (sub-sección C). Finalmente, **agrupar los documentos según su representación**, lo que proporciona el resultado final del problema (sub-sección D).

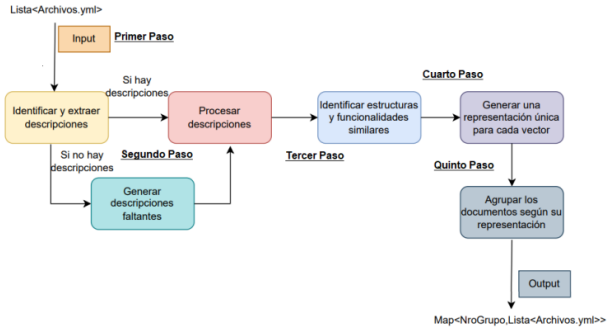


Fig. 1. Descripción gráfica de AMDO.

A continuación se explica a detalle cada una de las actividades del proceso junto a su configuración.

a. Identificar y extraer descripciones

Esta primera etapa consiste en la extracción de las diferentes descripciones que posee cada método perteneciente a cada especificación OpenAPI. Consta de recorrer todas las especificaciones y a medida que se extraen las descripciones ir agrupándolas en un mismo conjunto sin importar a qué especificación pertenecen. Cada endpoint de un microservicio posee información relevante que lo describe y en esa información se encuentra el apartado de la descripción.

Esta etapa es fundamental ya que las descripciones serán los datos que el algoritmo utiliza para luego poder vectorizar los archivos extrayendo las funcionalidades de las mismas. Se consideró las descripciones como entrada debido al gran aporte semántico que brindan, que puede ser tanto desde lo estructural como lo funcional [5].

Generar descripciones faltantes

Un problema usual es la falta de descripciones en las especificaciones de los documentos [6]. Muchas veces los programadores no las especifican o se utilizan herramientas que automatizan el proceso de creación de especificaciones OpenAPI que no generan descripciones para los diferentes métodos [6]. Es así que es necesario generar las descripciones faltantes para poder enriquecer los datos de entrada y así tratar de obtener un resultado final lo más preciso posible. Si bien existen diferentes métodos para la generación de descripciones, tienen el inconveniente de no generar descripciones lo suficientemente buenas o que para lograr descripciones aceptables demandan mucho tiempo y esfuerzo.

Por los problemas mencionados AMDO utiliza Chat GPT como herramienta complementaria para poder generar descripciones acorde a los endpoints que no la especifican. Chat GPT es una herramienta de generación de texto que utiliza Inteligencia Artificial y es capaz de mantener el contexto de la conversación [7]. Para ello, se creó un prompt donde se le pide al Chat GPT la generación de una descripción para un endpoint específico. Se define un prompt como una instrucción en lenguaje natural dada al chat. Este prompt ya viene por defecto en la herramienta y no es necesario modificarlo.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 27,2026 at 23:22:03 UTC from IEEE Xplore. Restrictions apply.

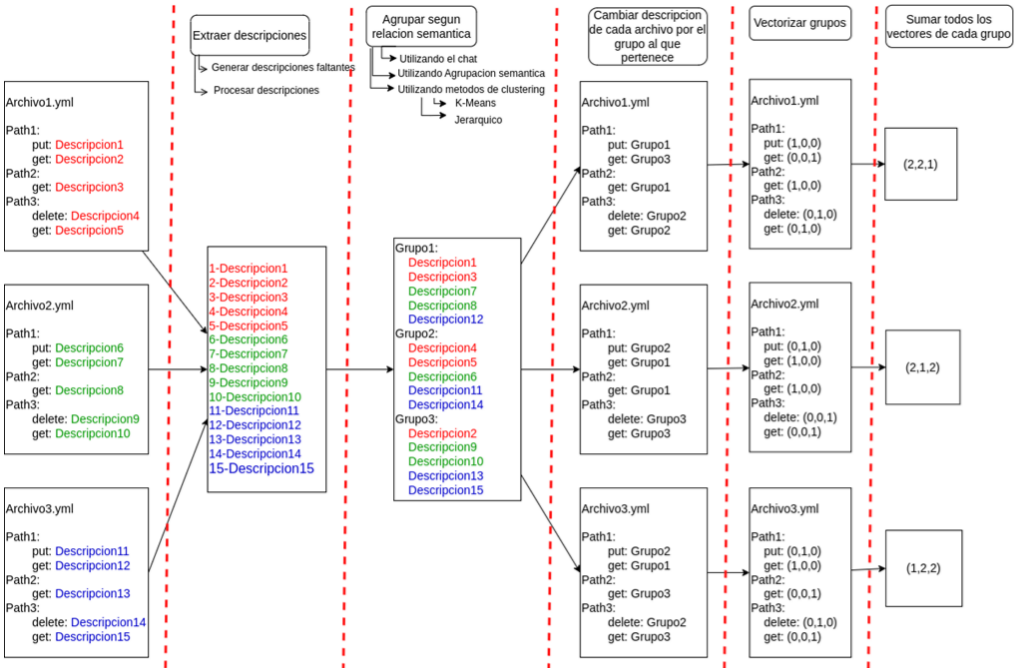


Fig. 2. Proceso de vectorización de los microservicios.

Procesar descripciones

Para lograr un mejor manejo de las descripciones y conseguir mejores resultados cuando se realizan los agrupamientos, es necesario procesar las descripciones para obtener las palabras más relevantes que la representan.

Para esto se utiliza la librería de Spacy. Spacy es una librería de software desarrollada en Python utilizada para el procesamiento de lenguaje natural [8]. Es una librería de alto rendimiento que hace que sea ideal para procesar grandes cantidades de texto de forma rápida y eficiente.

El procesamiento de las descripciones consiste, en primer lugar, en eliminar las stop words, que son palabras que no aportan información relevante al texto (pronombres, conjunciones y adverbios comunes) [9]. Luego, calcular la frecuencia de repetición de cada palabra en el texto y, por último, elegir las palabras más relevantes. La cantidad de palabras elegidas puede variar y ser un parámetro configurable.

B. Identificar estructuras y funcionalidades similares (IEF)

En esta instancia se busca agrupar las descripciones ya procesadas según su relación semántica. A este agrupamiento por sus siglas lo denominamos IEF para no confundirlo con el agrupamiento final que se explicará más adelante. Es necesario aclarar que para el agrupamiento no es importante a qué documento pertenece cada descripción, sino que se ven todas las descripciones en un mismo conjunto. El IEF permite asignarle a cada descripción un grupo el cual representará características funcionales y estructurales de los endpoints agrupados.

Se utilizaron diferentes métodos de agrupamiento para poder establecer el más óptimo y poder hacer una evaluación más precisa del resultado final del algoritmo. Se utiliza Chat GPT como estrategia de agrupamiento para así tratar de aprovechar su capacidad semántica y poder evaluar cómo se comporta al utilizar el entrenamiento que posee para resolver problemas más complejos que generar texto.

En la implementación, al utilizar el Chat GPT, el nombre de los grupos debe ser representativo a los elementos que se están agrupando. Cada descripción perteneciente a una operación tiene una relación (N, 1) con los grupos, es decir que cada grupo puede tener varias descripciones pero cada descripción sólo pertenece a un único grupo. Por ejemplo, en la Figura 3 se puede observar cómo las descripciones 1 y 3 están asociadas al Grupo 1, la descripción 2 al Grupo 3 y las descripciones 4 y 5 al Grupo 2.

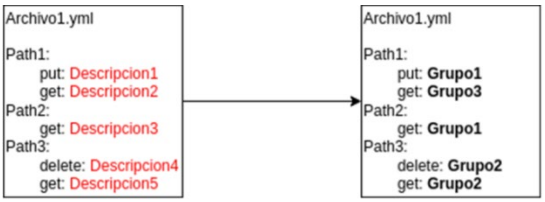


Fig. 3. Mapeo de las descripciones con los grupos a las que pertenecen.

Para realizar esta actividad se utilizan dos métodos: Agrupamiento utilizando solo Chat GPT y Agrupamiento utilizando Chat GPT y Spacy de manera asistida.

Método 1: Agrupamiento utilizando solo Chat GPT

La idea de este método es aprovechar la capacidad de procesamiento y generación de texto que tiene el Chat GPT para formar agrupamientos de las descripciones según la relación semántica de las mismas. El objetivo es que el Chat GPT relacione las diferentes descripciones para formar grupos precisos que permitan ser lo suficientemente descriptivos para las descripciones que agrupan.

Para tal fin, se genera un *prompt* que le indica al Chat GPT que debe agrupar los textos según su relación semántica, seguido de todas las descripciones. Este prompt, al igual que la utilizada en la generación de descripciones, ya está establecido por defecto en la herramienta y es fijo.

Se debe tener en cuenta que la API del Chat GPT tiene ciertas restricciones como es el límite de tokens en una consulta por lo que es necesario realizar múltiples consultas [10]. La cantidad de tokens es la capacidad que tiene el Chat GPT para manejar el contexto de la conversación [11]. Es así que, si la cantidad de descripciones es demasiado grande, será necesario realizar múltiples consultas donde, para poder mantener el contexto de las consultas previas, se pasa además del prompt los grupos ya creados para que intente

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 27,2026 at 23:22:03 UTC from IEEE Xplore. Restrictions apply.

agrupar con esos grupos y, de ser necesario, crear nuevos para las descripciones que no se relacionan con ninguno.

Método 2: Agrupamiento utilizando Chat GPT y Spacy de manera asistida

Con este método se busca simplificar las múltiples consultas al Chat GPT para que sólo realice la tarea de agrupar en nuevos grupos, es decir, realizar consultas independientes sin la necesidad de tener que mantener un contexto con las consultas previas.

Este método consiste en ir guardando los grupos que se van creando en cada consulta previamente a realizar una nueva consulta. Con Spacy se corrobora si alguna de las descripciones puede pertenecer a alguno de los grupos ya creados. Spacy permite, con el método "similarity", verificar si dos textos tienen relación semántica dando un valor entre 0 y 1 [12]. Entonces por cada descripción se consulta a cada grupo ya creado cuál es su similitud y se queda con la más grande. Si este valor obtenido supera cierto "umbral", entonces se considera que la descripción pertenece a ese grupo. Por ejemplo, si ya se crearon 3 grupos y el umbral que se está utilizando es de 0,7 y que la descripción a evaluar tiene una relación semántica de 0,4 con el grupo 1, 0,7 con el grupo 2 y 0,85 con el grupo 3. Entonces, en este caso, se puede verificar que el mayor valor obtenido es el de 0,85 y supera el umbral de 0,7 siendo así que entonces se agrupa con el grupo 3.

Luego, las descripciones que no pertenecen a ningún grupo por no superar el umbral, son agrupadas como se explicó en el método anterior. Es así que siempre se deberían formar nuevos grupos en cada consulta ya que se sabe que no pertenecen a los ya creados. Para esto es necesario que el Chat GPT genere un nombre de grupo representativo a las descripciones que agrupa para así poder realizar el agrupamiento manual. Si bien de esta forma se minimiza el problema de agrupar descripciones que deberían estar juntas, el problema puede seguir sucediendo al realizar las consultas individuales. Es así que este nombre de grupo se utilizará en la siguiente etapa para crear vectores más precisos que tengan información de cómo se relaciona cada grupo con el resto de grupos.

C. Generar una representación única para cada documento

Una vez agrupadas las descripciones en grupos, cada grupo puede ser expresado como un vector que tiene un "1" en la posición correspondiente al grupo.

Grupo1: (1,0,0)

Grupo2: (0,1,0)

Grupo3: (0,0,1)

De esta forma, cada documento OpenAPI puede describirse como un vector formado por la sumatoria de cada uno de los vectores que lo contiene. Por ejemplo, en la Figura 4 se observa cómo quedaría el documento 1 vectorizado donde en primer lugar se reemplaza el grupo por el vector al que pertenece y luego se realiza la sumatoria de esos vectores.

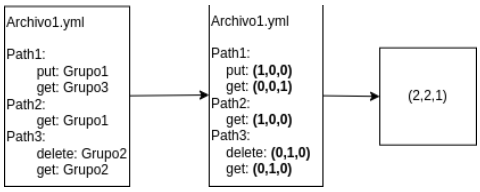


Fig. 4. Generación de un vector para un microservicio según su funcionalidad y estructura.

$$(1,0,0) + (0,0,1) + (1,0,0) + (0,1,0) + (0,1,0) = (2,2,1)$$

En este caso, en la Figura 4, el vector final describe que la documentación tiene 2 operaciones relacionadas al grupo 1, 2 operaciones relacionadas al grupo 2 y 1 operación relacionadas al grupo 3.

Entonces utilizando la librería Spacy con el método "similarity" de la misma forma que se utilizó en el método de agrupamiento 2, se calcula la relación semántica de los grupos. Por ejemplo, si el grupo 1 tiene una relación semántica de 0,8 con el grupo 2 y de 0,3 con el grupo 3 entonces ahora el vector en vez de ser (1,0,0) va a ser (1, 0.8, 0.4). De esta forma ahora dos grupos que tienen descripciones que se relacionan formarán vectores similares o cercanos en el espacio.

D. Agrupar los documentos

Dado que la representación única de los documentos está dada por vectores se puede reutilizar algún algoritmo de clustering. En este método primero se realiza una vectorización de las descripciones y luego con esos vectores se aplica el algoritmo. En este caso los documentos ya se encuentran vectorizados por lo que no se realiza la primera etapa del método sino que solo se utiliza el algoritmo de clustering para formar los grupos deseados. El objetivo principal es que los datos de un mismo grupo o cluster sean similares entre sí y diferentes al resto de grupos o clusters.

El método de clustering utilizado por AMDO es k-means ya que es utilizado en el análisis de datos debido a sus diversas ventajas [13]. En primer lugar, es relativamente fácil de implementar, lo que lo convierte en una herramienta accesible incluso para aquellos que no poseen una amplia experiencia en análisis de datos. Esto se debe a su enfoque simple y directo en la segmentación de datos en grupos o clusters. Adicionalmente, tiene buena capacidad para analizar grandes conjuntos de datos es una de sus principales ventajas, ya que puede manejar tamaños de datos que serían prohibitivos para otros enfoques.

IV. DISEÑO DEL EXPERIMENTO

Para poder realizar un análisis completo de los resultados obtenidos y con el objetivo de evaluar la herramienta se formularon las siguientes preguntas de investigación.

P1- ¿Qué técnica es mejor para identificar relaciones semánticas?

P2- ¿Completar descripciones con Chat GPT ayuda a obtener mejores resultados?

P3- ¿Cuál k es mejor para dar como entrada al algoritmo de clustering?

P4- ¿Existen diferencias significativas en los resultados de agrupamiento final entre los métodos utilizados?

P5- ¿Qué método logró mejores tiempos?

A. Caso de estudio

Se utiliza como caso de estudio la API de Twilio que proporciona servicios de comunicación en la nube [15]. La Twilio API permite a los desarrolladores integrar servicios de voz, mensajes de texto (SMS), video y otros tipos de comunicación en aplicaciones y sitios web. Los desarrolladores pueden utilizar Twilio para enviar y recibir mensajes de texto, realizar llamadas telefónicas, gestionar conferencias, entre otras funcionalidades de comunicación. Esta API consta de cuarenta y cinco microservicios los cuales se utilizan para analizar el funcionamiento de la herramienta. Además, se realizó un agrupamiento manual de

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 27,2026 at 23:22:03 UTC from IEEE Xplore. Restrictions apply.

los microservicios la cual se considera como opinión experta para poder comparar los diferentes resultados.

B. Métricas utilizadas

Para poder realizar una correcta evaluación de los resultados de los experimentos se utilizan diferentes métricas. Por un lado se define un grado de éxito y de fallo que indica qué tan bien se agrupan los elementos según un criterio experto. Por otro lado, se utiliza la medida de silhoutte que indica qué tan bien se agrupan los elementos según la distancia que tienen su representación vectorial. Estas métricas son definidas y explicadas a continuación.

1) Grado de éxito y grado de fallo

El agrupamiento experto se emplea para evaluar los agrupamientos generados por el algoritmo en función de la configuración utilizada. Para evaluar los agrupamientos del algoritmo, se compara cada elemento con su agrupamiento tanto en el algoritmo como en la opinión del experto. Se realiza la intersección entre ambos grupos, lo que proporciona dos valores clave que son las primeras métricas a utilizar, el grado de éxito y el grado de fallo.

Grado de éxito: Con cuántos elementos coincide un elemento que se encuentra agrupado por el algoritmo en comparación de cómo se encuentra agrupado por el experto.

Grado de fallo: Con cuántos elementos no coincide un elemento que se encuentra agrupado por el algoritmo en comparación de cómo se encuentra agrupado por el experto.

Cabe aclarar que estas métricas presentan valores entre 0 y 1, siendo valores cercanos al 1 considerados como valores altos. En contraparte, los valores cercanos a 0 se consideran bajos.

Entonces para que un agrupamiento hecho por el algoritmo se considere óptimo se busca maximizar el grado de éxito y minimizar el grado de fallo. También hay que tener en cuenta el número de grupos que se está comparando. Cuando el número de grupos hecho por el algoritmo es mayor al número de grupos del experto, el grado de fallo tenderá a disminuir. Mientras que si el número de grupos del algoritmo es menor, entonces el grado de éxito tenderá a aumentar.

Una vez obtenido el grado de éxito y fallo de cada elemento se obtiene con el promedio de los mismos el grado de éxito y fallo final del método.

Grado de éxito final: Promedio del grado de éxito de los elementos agrupados

Grado de fallo final: Promedio del grado de fallo de los elementos agrupados

En resumen, el grado de éxito y de fallo final se utiliza para verificar qué tan bien se creó cada grupo según la opinión experta.

2) Medida de silhouette

Para evaluar la calidad de los clusters, se utiliza el coeficiente de Silhouette, el cual se calcula para cada punto de datos y se basa en la diferencia entre la cohesión y la separación [14].

La cohesión se refiere a la proximidad de los individuos dentro de un cluster [16]. Mide qué tan cerca están los puntos dentro de un cluster unos de otros. Cuanto más cercanos estén los puntos en un cluster, mayor será la cohesión. La cohesión se puede calcular, por ejemplo, tomando el promedio de las distancias entre todos los puntos dentro de un cluster. Por otro lado, la separación se refiere a la distancia entre los clusters [16]. Mide qué tan diferente es un cluster de los clusters vecinos. Cuanto mayor sea la distancia entre los clusters, mayor será la separación. La separación se puede calcular, por ejemplo, tomando el promedio de las distancias entre los puntos de un cluster y los puntos del cluster más cercano que no lo contiene.

Coeficiente de Silhouette = (Separación - Cohesión) / max(Separación, Cohesión)

El valor de Silhouette para un conjunto de datos es el promedio de los valores de Silhouette para todos los puntos. En general, un valor de Silhouette cercano a 1 indica que los puntos están bien asignados a sus clusters y que los clusters están bien separados. Un valor cercano a 0 sugiere que los puntos pueden estar en la frontera de dos clusters, y un valor negativo indica una mala asignación de puntos a clusters.

V. Resultados Obtenidos

Dado que el agrupamiento experto tiene un número de clusters (k) igual a 15, se realizaron diferentes escenarios de prueba respetando este número de clusters finales. Los modelos presentados utilizan diferentes configuraciones iniciales en cuanto al método intermedio de agrupamiento de descripciones. Además, cada escenario es evaluado con los dos métodos de clustering *mencionados*. Adicionalmente, en cada escenario se evalúa el impacto de generar o no las descripciones utilizando el *Chat GPT*. La Tabla 2 muestra los diferentes escenarios evaluados con las métricas de éxito y el fallo del agrupamiento.

Escenario 1: Modelo utilizando Chat GPT como método intermedio de agrupamiento.
Escenario 2: Modelo utilizando agrupamiento semántico como método de agrupamiento intermedio.

La medida de silhouette se utiliza para analizar diferentes “k” y así obtener cual es el mejor número de clusters finales en la fase final del nuestro enfoque. Dado los resultados obtenidos en la Tabla 1 es posible realizar diferentes análisis con cada uno de los escenarios, los cuales son detallados a continuación.

Tabla 1. Análisis de los escenarios según las métricas de éxito y fallo.

k = 15			Con descripciones generadas			Sin descripciones generadas		
Escenario	**IEF (Agrupamiento intermedio)**	**Agrupamiento final**	**Éxito**	**Fallo**	**Tiempo (seg)**	**Éxito**	**Fallo**	**Tiempo (seg)**
1	Chat GPT	K-Means	0.64	0.4	5585.24	0.47	0.6	1813.89
2	Semántico	K-Means	0.51	0.55	98.56	0.59	0.62	60.66

3) Escenario 1

Este escenario utiliza Chat GPT para realizar el agrupamiento intermedio. Como se puede observar en la Tabla 2, se logró conseguir un alto grado de éxito y un bajo grado de fallo al generar descripciones faltantes. En

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 27,2026 at 23:22:03 UTC from IEEE Xplore. Restrictions apply.

contraparte, los resultados se invierten al no generar descripciones, el grado de éxito es menor al grado de fallo.

Por otro lado, se destaca que este escenario es el que mejores resultados obtuvo por lo que se utiliza para evaluar diferentes números de clusters finales (k). Para eso se utiliza la medida de silhoutte como se mencionó previamente. En la Tabla 2 se pueden observar medidas similares en el score final siendo el mayor score obtenido con un k igual a 5. Es decir, que se obtuvo un mejor agrupamiento con este número de cluster final.

TABLA 2. ANÁLISIS DE LA MEDIDA DE SILHOUTTE CON DIFERENTES CLUSTERS UTILIZANDO CHAT GPT COMO MÉTODO INTERMEDIO Y K-MEANS COMO ALGORITMO DE CLUSTERING.

k	Score final
5	0.55
10	0.49
15	0.49
20	0.50

4) Escenario 2

Por último, utilizando el agrupamiento semántico, al igual que el escenario anterior, no logró los resultados deseados y no se puede apreciar una diferencia con la generación de descripciones activada. Este método no relaciona los grupos entre sí al generar el agrupamiento de las descripciones lo que, por un lado, podría indicar que no se generan de forma correcta ya que podría haber relaciones entre grupos. El valor de umbral utilizado en este método podría influir en qué tan exactos son estos agrupamientos y queda como trabajo futuro el estudio de diferentes configuraciones utilizando este umbral.

VI. DISCUSIÓN

Este apartado tiene como objetivo responder las preguntas planteadas en la Sección II.

Respecto a la P1, el escenario con ChatGPT presenta mayor éxito (0.64) y menor fallo (0.40) en comparación con el escenario sin generación de descripciones, aunque con un tiempo de procesamiento significativamente mayor. Esto muestra que ChatGPT mejora la precisión del agrupamiento, aunque a un mayor costo computacional. Para la P2, generar descripciones con ChatGPT mejora los resultados en todos los escenarios, especialmente en el primero, donde revierte los malos resultados obtenidos sin esta generación. En cuanto a la P3, se evaluaron diferentes valores de k, y el valor óptimo fue k = 5, según la métrica de Silhouette, lo que sugiere una buena cohesión y separación entre grupos. La P4 se responde con esta misma observación: menos clusters pueden mejorar la organización sin comprometer la calidad. Finalmente, sobre la P5, los métodos más rápidos fueron los semánticos y de clustering tradicionales, mientras que el uso de ChatGPT resultó el más costoso en tiempo debido a las múltiples consultas a la API.

VII. LIMITACIONES

Uno de los principales desafíos en la agrupación semántica de microservicios es interpretar correctamente el significado de las descripciones de los endpoints, las cuales suelen ser incompletas, inconsistentes o ambiguas. Esto dificulta que los modelos de lenguaje identifiquen con precisión similitudes semánticas, especialmente cuando los términos varían según el contexto. Además, la escalabilidad representa otro obstáculo, ya que a medida que crece el número y la complejidad de los servicios, el análisis se vuelve más costoso computacionalmente.

Si bien modelos como ChatGPT ayudan a generar descripciones faltantes y mejorar el agrupamiento, también pueden introducir ruido cuando producen textos irrelevantes o imprecisos. Las descripciones generadas no siempre reflejan fielmente la funcionalidad real del microservicio, lo que limita la eficacia del agrupamiento basado solo en texto.

Para superar estas limitaciones, se sugiere complementar el análisis semántico con enfoques estructurales y técnicas híbridas que incorporen patrones de uso. Además, la validación manual de las descripciones generadas podría mejorar la calidad del agrupamiento. En síntesis, aunque los modelos de lenguaje aportan beneficios, aún existen desafíos importantes que requieren soluciones combinadas para lograr agrupamientos más precisos y escalables.

VIII. CONCLUSIÓN

Este trabajo presentó AMDO (Agrupamiento de Microservicios en la Documentación OpenAPI), una propuesta que combina el análisis semántico y estructural de especificaciones OpenAPI con modelos de lenguaje como ChatGPT y técnicas de clustering como k-means. AMDO permite automatizar el proceso de agrupamiento de microservicios, facilitando su organización, mantenimiento y despliegue en arquitecturas distribuidas.

Los resultados obtenidos muestran beneficios concretos, como una mayor coherencia en los grupos generados y una reducción en el esfuerzo manual requerido. En particular, el uso de ChatGPT para generar descripciones faltantes enriqueció la documentación disponible, mejorando la precisión del agrupamiento. No obstante, se identificaron desafíos relacionados con la calidad de las descripciones, la ambigüedad del lenguaje natural y la sensibilidad del enfoque a datos incompletos.

Como líneas futuras, se propone fortalecer el proceso de agrupamiento intermedio, explorar configuraciones dinámicas del umbral semántico y desarrollar un mecanismo automático para determinar el número óptimo de clusters. Estas mejoras permitirán aumentar la robustez y escalabilidad de AMDO, ampliando su aplicabilidad en escenarios reales de desarrollo y operación de sistemas basados en microservicios.

REFERENCIAS

[1] L. Baresi, M. Garriga, “Microservices: The evolution and extinction of web services?”, *Microservices: Science and Engineering*, 3-28, 2020. DOI: https://doi.org/10.1007/978-3-030-31646-4_1

[2] C. Pautasso, O. Zimmermann, F. Leymann, “Restful web services vs." big web services: making the right architectural decision”, in *2008 17th international conference on World Wide Web*, pp. 805-814. DOI: https://doi.org/10.1145/1367497.1367606

[3] A. Tzavaras, N. Mainas, E.G. Petrakis, E. G. “OpenAPI framework for the Web of Things”, *Internet of Things*, 21, 100675, 2023. DOI: https://doi.org/10.1016/j.iot.2022.100675

[4] T. D. Stojanovic, S. D. Lazarevic, M. Milic, I. Antovic, “Identifying microservices using structured system analysis”, in *2020 24th IEEE international conference on information technology (IT)*, pp. 1-4. DOI: 10.1109/IT48810.2020.9070652.

[5] L. Baresi, M. Garriga, and A. De Renzis.”Microservices Identification Through Interface Analysis”, in *2017 6th Service-Oriented and Cloud Computing European Conference*, pp. 19-33. DOI: 10.1007/978-3-319-67262-5_2

[6] L. Da Rocha Araujo, G. Rodríguez, S. Vidal, C. Marcos, and R. Pereira dos Santos, R. “Empirical Analysis on OpenAPI Topic Exploration and Discovery to Support the Developer Community”, *Computing and Informatics*, 2021. DOI: https://doi.org/10.31577/cai_2021_6_1345

[7] H. Saeidnia, “Open AI, ChatGPT: To Be, or Not to Be, That Is the Question”, *Information Matters*, 3, 6, 2023. DOI: http://dx.doi.org/10.2139/ssrn.4478630.

[8] A. Fantechi, S. Gnesi, S. Livi, L. Semini, “A spaCy-based tool for extracting variability from NL requirements”, in *2021 25th ACM*

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 27,2026 at 23:22:03 UTC from IEEE Xplore. Restrictions apply.

International Systems and Software Product Line Conference, B, pp. 32-35. DOI: https://doi.org/10.1145/3461002.347307.

[9] W. J. Wilbur, K. Sirotkin, "The automatic identification of stop words", *Journal of Information Science*, 18(1), 45-55, 1992. DOI: https://doi.org/10.1177/01655515920180010.

[10] J. Xian, T. Teofili, R. Pradeep, J. Lin, "Vector search with OpenAI embeddings: Lucene is all you need", in *2024 17th ACM International Conference on Web Search and Data Mining*, pp. 1090-1093. DOI: https://doi.org/10.1145/3616855.3635691

[11] N.B. Korade, M. B. Salunke, A.A. Bhosle, P.B. Kumbharkar, G.G. Asalkar, R.G. Khedkar, "Strengthening Sentence Similarity Identification Through OpenAI Embeddings and Deep Learning", *International Journal of Advanced Computer Science & Applications*, 15(4), 2024. DOI: 10.14569/IJACSA.2024.0150485.

[12] Z. Pauzi, A. Capiluppi, "Text similarity between concepts extracted from source code and documentation", in *2020 21st International Conference in Intelligent Data Engineering and Automated Learning (IDEAL)*, pp. 124-135. DOI: 10.1007/978-3-030-62362-3_12.

[13] M. K. Gysel, L. Kolbener, W. Giersche, O. Zimmermann, "Service cutter: A systematic approach to service decomposition", in *2016 European Conference on Service Oriented and Cloud Computing*, pp.185–200. DOI: 10.2140/pjm.1953.3.369

[14] M. Shutaywi, N.N Kachouie, "Silhouette analysis for performance evaluation in machine learning with applications to clustering", *Entropy*, 23(6), 759, 2021. DOI: https://doi.org/10.3390/e23060759

[15] J. Ahn, J. Min, H.T. Lee, J. Paek, "Unified Framework for End-User Authentication Protocol in Feature-as-a-Service Models", in *2023 14th IEEE International Conference on Information and Communication Technology Convergence (ICTC)*, pp. 539-542. DOI: 10.1109/ICTC58733.2023.10393074

[16] Y. Liu, Z. Li, H. Xiong, X. Gao, J. Wu, "Understanding of internal clustering validation measures", in *2010 IEEE international conference on data mining*, pp. 911-916. DOI: 10.1109/ICDM.2010.35.

[17] O. Al-Debagy, P. Martinek, "A New Decomposition Method for Designing Microservices", *Periodica Polytechnica Electrical Engineering and Computer Science*, 63(4), 2019. DOI: https://doi.org/10.3311/PPee.13925

[18] C. Schröer, F. Kruse, J. Marx Gómez, "A Qualitative Literature Review on Microservices Identification Approaches", *Communications in Computer and Information Science, Springer*, 1310, 2020. DOI: https://doi.org/10.1007/978-3-030-64846-6_9.

[19] G. Rodriguez, A. Teyseyre, A. Soria, L. Berdun, "A Visualization Tool to Detect Refactoring Opportunities in SOA Applications", in *2017 XLIII IEEE Latin American Computer Conference (CLEI)*, pp. 1-10. DOI: 10.1109/CLEI.2017.8226440

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 27,2026 at 23:22:03 UTC from IEEE Xplore. Restrictions apply.